

题目 1：多进程任务分发与状态监控

请编写一个程序，父进程创建 3 个并发执行的子进程，分别执行以下任务：

- (1) 子进程 1：生成 10 个随机数并保存到文件 `data.txt`；
- (2) 子进程 2：从 `data.txt` 中读取数据并计算平均值；
- (3) 子进程 3：每隔 2 秒输出系统当前时间。

父进程需等待所有子进程结束，并输出每个子进程的退出状态（正常退出或获得终止信号退出）。如果父子进程间存在同步问题，请处理这些同步问题以确保任务的正确执行。

在编写代码时可能会用到以下定义及函数：

`sem_t s = 0;` // 定义一个信号量 `s`，并对其赋初值为 0

`sem_wait(s);` // 对信号量 `s` 执行 P 操作，若 `s` 的值 < 0 ，则阻塞调用该函数的进程

`sem_post(s);` // 对信号量 `s` 执行 V 操作，若 `s` 的值 ≤ 0 ，则唤醒等待该信号量的进程

`fork();` // 创建一个子进程，若是父进程则返回子进程的 `pid`，若是子进程则返回 0

`waitpid(pid, &status, 0);` // 父进程等待指定 `pid` 的子进程结束，根据 `status` 的值了解是正常退出，还是被信号终止。

// 子进程 1：生成随机数并写入文件（5 分）

```
void child1_task() {  
    // 写入数据  
    FILE *fp = fopen(DATA_FILE, "w");  
    for (int i=0; i<10; i++)  
        fprintf(fp, "%d\n", rand()%100);  
    fclose(fp);  
  
    // 释放信号量通知子进程 2  
    sem_post(sem);  
    exit(EXIT_SUCCESS);  
}
```

// 子进程 2：读取数据并计算平均值（5 分）

```
void child2_task() {  
    // 等待信号量阻塞直到子进程 1 完成  
    sem_wait(sem);  
  
    // 读取数据并计算
```

```

FILE *fp = fopen(DATA_FILE, "r");
int sum=0, num;
while (fscanf(fp, "%d", &num) == 1) sum += num;
printf("Average: %.2f\n", (double)sum/10);
fclose(fp);
exit(EXIT_SUCCESS);
}

```

// 子进程 3：定时输出系统时间 （5 分）

```

void child3_task() {
    for (int i = 0; i < 5; i++) {
        time_t now = time(NULL);
        struct tm *t = localtime(&now);
        printf("[PID %d] 当前时间: %04d-%02d-%02d %02d:%02d:%02d\n",
            getpid(), t->tm_year+1900, t->tm_mon+1, t->tm_mday,
            t->tm_hour, t->tm_min, t->tm_sec);
        sleep(2);
    }
    exit(EXIT_SUCCESS);
}

```

```

int main() {
    pid_t pids[3];
    int status;

    // 创建三个子进程 （4 分）
    for (int i = 0; i < 3; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            switch(i) {
                case 0: child1_task(); break;
                case 1: child2_task(); break;
                case 2: child3_task(); break;
            }
        } else if (pid > 0) {
            pids[i] = pid;
        } else {
            perror("fork 失败");
            exit(EXIT_FAILURE);
        }
    }
}

```

// 等待所有子进程结束 （4 分）

```
for (int i = 0; i < 3; i++) {  
    waitpid(pids[i], &status, 0);  
    printf("子进程%d (PID %d) 状态: ", i+1, pids[i]);  
  
    if (status==0) {  
        printf("正常退出);  
    } else if (status==1)) {  
        printf("被信号终止);  
    }  
}  
return 0;  
}
```


题目 2：实时视频流处理系统的调度策略设计 （25 分）

1. 调度算法选择与混合策略 （共 16 分）

需采用抢占式优先级调度（实时任务）+ 多级反馈队列（非实时任务）的混合算法，原因如下：

- 实时编码任务：采用严格的优先级抢占调度算法，截止时间越早者越优先，确保任务在 10ms 内开始执行。
（5 分）
- 非实时转码任务：高吞吐量与公平性需求适合采用多级反馈队列（MLFQ）调度算法，通过动态降级长任务、提升短任务优先级，平衡吞吐与公平。
（5 分）

算法分层	规则与指标	技术细节
实时任务层	- 优先级：截止时间越近，优先级越高（10ms 为阈值） （3 分） - 调度方式：全局抢占式 （2 分）	- 实时任务到达时插入优先队列，触发抢占检查 - 若队列满，丢弃最旧任务并告警
非实时任务层 （MLFQ）	- 分 3 级队列： （2 分） Q0（5-15ms 任务） Q1（15-30ms） Q2（30-50ms） - 时间片： Q0=5ms，Q1=10ms，Q2=20ms - 降级规则：任务超时后降级至下一级队列 - 优先级提升：每 60 秒重置所有任务至 Q0，避免长任务饥饿	- 短任务（5ms）在 Q0 快速完成，长任务（50ms）逐步降级至 Q2 - Q2 采用轮转调度，保证长任务最终完成 调度细节（3 分）

关键指标：实时任务：响应延迟≤10ms

2. 长任务阻塞问题的解决 （5 分）回答其中一种即可得分

- (1) 采用优先级抢占的方式，可以确保实时任务绝对优先。
- (2) 实时任务与非实时任务相分离，为实时任务预留比如 20%的 CPU 和带宽资源，确保突发流量下，即使非实时任务占满了比如 80%的资源，实时任务仍可处理。

题目 3：自动驾驶车队协同系统

请设计一个包含 1 辆头车 (Leader) 和 N 辆跟随车辆 (Follower) 的自动驾驶车队协同系统，需满足以下同步条件：

- 头车用 `broadcast_path()` 发送当前路径数据后，跟随车才能启动路径跟踪。
- 当所有跟随车都执行了路径规划任务 `execute_task()` 后，头车才能发送下一指令。

1. 请用信号量机制来解决头车和跟随车的同步问题。需说明设置的信号量的含义及初值。

2. 若跟随车出现故障，你的程序会出现什么问题？可以如何改进？

三、参考答案：

同步逻辑分析

1. 头车与跟随车的同步需求：

- 头车发送路径数据后，跟随车才能启动 → **头车释放信号量通知跟随车**
- N 辆跟随车执行路径规划并就绪后，头车才能发送下一指令 → **头车需等待 N 次信号量确认**

2. 信号量设计：

- path_ready：初始为 0，头车发送路径数据后释放，跟随车等待该信号量启动。
- followers_done：初始为 0，跟随车就绪后释放，头车通过 sem_wait 操作累积 N 次确认。

//全局信号量与共享变量

sem_t path_ready; //路径数据就绪信号量（初始 0）（2 分）

sem_t followers_done; //跟随车就绪信号量（初始 0）（2 分）

//头车线程（1 分）

```

void leader() {
    while(1) {
        //1. 规划并发送路径数据 (1 分)
        broadcast_path();

        //2. 释放 N 次 path_ready, 允许 N 辆跟随车启动 (3 分)
        for (int i=0; i<current_active; i++) {
            sem_post(&path_ready);
        }

        //3. 行驶 (1 分)
        execute_task();

        //4. 等待所有跟随车就绪 (需动态容错) (3 分)
        int count = current_active;
        while (count > 0) {
            sem_wait(&followers_done);
            count--;
        }
    }
}

```

//跟随车线程

```

void follower(int id) {
    while(1) {
        //1. 接收头车发送路径数据 (1 分)
        recv_path();

        //2. 等待头车发送 path_ready 通知, 以启动行程 (2 分)
        sem_wait(&path_ready);

        //3. 行驶 (1 分)
        execute_task();

        //4. 通知头车已就绪 (2 分)
        sem_post(&followers_done);
    }
}

```

当出现一辆故障跟随车时，头车会被阻塞在 `sem_wait(&followers_done)` 处，可考虑的方法：

- 1) 需要设置检查程序，当发现跟随车出现故障时，执行 `sem_post(&followers_done)` 将头车唤醒，并将 `current_active` 减 1。（3 分）
- 2) 头车将 `sem_wait(&followers_done)` 改为 `sem_timedwait(&followers_done)`，为阻塞设置超时，若发生超时，则将 `current_active` 减 1。（3 分）

题目 4：医院病人排队调度系统 （25 分）

- 1) 调度函数 `schedule()` 已经写好，其中的关键步骤 `pickNext()` 需要你来填写。
- 2) 调度可能在三种情况下发生：当前病人完成治疗、时间片结束、新病人到来，三种情况分别对应 `onPatientLeaving()`、`onExpiration()`、`onPatientArrival()`，后两个函数需要你填写。

// 执行调度

```
void schedule() {
```

```
    Patient *next = pickNext();
```

```
    if (next->remaining_time > 0){ // 如果 remaining_time 为 0，表示病人来自 Q2，则
```

不设置时间片

```
        setTimeSlice(next->remaining_time);
```

```
        switchTime = getTime();
```

```
        switchPatient(getCurrentPatient(), next);
```

```
    }
```

```
}
```

参考答案：

void onExpiration() { (6分)

```
Patient *current = getCurrentPatient();
current->remaining_time = 0;
if (current->type == 'A') {
    enqueue(Q0, current);
} else {
    enqueue(Q2, current);
}
schedule();
}
```

void onPatientArrival(Patient *p) { (共12分，每个入队规则4分)

```
if (p->type == 'A') {
    enqueue(&Q0, p);
    if (queueSize(&Q0) > 3)
        告警();
} else {
    enqueue(&Q1, p);
}
// 判断是否需要抢占
Patient *current = getCurrentPatient();
if ((p->type == 'B' && current->type == 'B' && current->remaining_time == 0) ||
    (p->type == 'A' && current->type == 'B')) {
    // 执行抢占
    if (current->remaining_time == 0) { // 当前病人来自
Q2
        enqueueFront(&Q2, current);
    } else { // 当前病人来自Q1
        current->remaining_time -= getTime() - switchTime;
        enqueueFront(&Q1, current);
    }
    schedule(); // 实际上必然切换到p
}
}
```

Patient *pickNext() { (7分)

```
Patient *p;
p = dequeue(&Q0);
if (p != NULL) {
    p->remaining_time = Q0.time_slice;
    return p;
}
p = dequeue(&Q1);
if (p != NULL) {
    if (p->remaining_time == 0)
        p->remaining_time = Q1.time_slice;
    return p;
}
p = dequeue(&Q2);
return p;
}
```